Theta 4 Verification Exercise

André Teixeira Pino
atp1n24@soton.ac.uk
University of Southampton
ELEC2301
April 30, 2026

I. Abstract

Design verification is the process of testing and validating the functionality of a digital design before its deployment or release. In this report a given FIFO buffer designed externally is verified with a UVM style testbench. It is tested against its specification and corner cases and its outputs are checked with a reference model in a scoreboard. The functional coverage for the simulation was 98.07% with errors found in the empty flag logic and read of the buffer after a reset.

II. Introduction

Design verification is the process of testing and validating the behaviour of a specific digital design before its deployment or release, with the goal of eliminating errors and bugs and ensuring that the system performs as expected and meets the required specifications. It is one of the most crucial tasks in the development of digital systems [1].

This report is an introduction to the task of verification, with the sole purpose to verify a provided FIFO buffer designed externally against its specification and corner cases, verify correctness and completeness and understand where things could have gone wrong. The design itself will not be fixed, but its errors will be pointed out together with potential causes for them. The verification of the buffer will only be considered successful and thorough if the performed tests get a high coverage (greater than 95%) and the DUT will only pass the test if the simulation finds no fails.

III. Background

This section contains a summary of the FIFO's specification and introduces many relevant topics for the testbench.

The design under test (DUT) is a FIFO buffer. According to its specification, it is clocked by a single posedge clock (clk) and contains a synchronous active low reset signal (n_rst) that only initialises the pointers inside the buffer. It is parametrised for its bit width and depth. For its inputs, it has write (we) and read (re) enable signals, together with the write data input (wrdata). The FIFO has five total outputs: four status flags and the read data output (rddata). Its status flags are full, empty, almost_full and almost_empty. The last two are set with respect to parametrised thresholds. When the number of entries is higher or lower than the defined threshold, the flag is set. When full is set no data can be written to the FIFO and when empty is set, read data is not defined. The FIFO addresses should wrap around transparently and same cycle read and write operations are supported.

The relevant System Verilog constructs used in the code for the testbench are:

- **Mailbox:** SV construct that allows for 2 separate modules to communicate and send data between each other.
- **Interface:** SV construct that encapsulates signals into a block, making it easier to connect them between various modules. Used to connect to the DUT in verification tasks.
- **Queue:** SV construct that behaves like a dynamic array and contains built-in methods for dynamic insertion and removal of entries.

A reference model used in a scoreboard is a golden design implementation that can be used to compare the DUT's outputs to and check correct functionality of the system.

Coverage is a simulation metric that measures the quality of a test by checking various metrics. Functional coverage is implemented by the user using covergroups and coverpoints and provides more meaningful results [1].

IV. Method

A. Testbench Architecture

This testbench followed an architecture that focuses in code reusability and modularity, separating stimulus generation, signal driving, monitoring and checking in order to improve clarity in the verification process. It is similar to the structure of a UVM testbench, although without using the actual library. It uses the OOP features and constructs of System Verilog to create an environment that runs tests on the DUV and monitors its outputs at the same time. This enables the testbench to perform any test on the design by simply generating the input signals and the rest is handled by the classes, even checking the results. Figure 1 shows a diagram with the architecure, including all used modules and how they interact with each other.

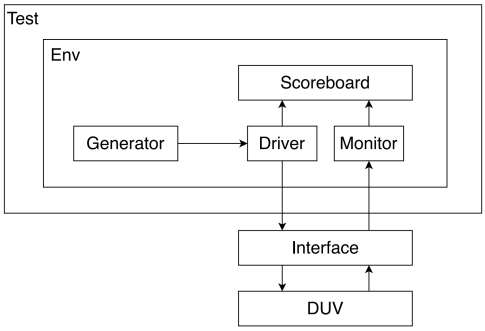


Fig. 1: Testbench Architecture Diagram [2]

The simulation loop starts in the generator module, referred to as sequencer in this report, where the input sequences for that cycle are generated. It sends the inputs to the driver module via a mailbox, and the driver simply receives them and sends the signals to the interface at the negedge of the clock to avoid conflicts with the posedge. The interface is the link between the ”virtual” world of the simulation and the ”physical” design under verification (DUV). When the DUT receives the signals, it generates the outputs which are monitored every clock cycle by the monitor class. This simply copies the information in the interface to a transaction item and sends it through a mailbox to the scoreboard module. The scoreboard with the information of the the input sequence and output generated by the DUV, updates its reference model and compares the outputs with the correct reference ones, asserting a pass or fail if any mismatch is found. All these modules are instantiated and connected in an environment class that performs all tests. The testbench then instantiates the actual FIFO and the simulation environment, generates the clock and distributes it to all modules. It also performs the first initial reset to the DUV.

B. Verification Strategy

In order to verify a design like this one, four main points were originally considered:

1) how to make sure that the design follows the specification correctly;
2) how to test corner cases to verify the functionality of the DUT even in extreme conditions;
3) how to cover as many cases as possible without the test becoming too long;
4) how to know when we can stop testing and provide a correct verdict about the functionality of the DUT;

Starting by the first point, to ensure that the specification was met, a reference model was created. This model was built using the queue type from System Verilog which perfectly maps to the requirements in the specification, but written in a much simpler way as it doesn’t need to be synthesisable code. Write and read operations are emulated by the push_back() and pop_front() methods, respectively, the FIFO’s internal count uses the size() method and the resets behaviour is emulated by the delete() method. The reference model keeps track of the correct behaviour and state of the design for the test being performed, and for a FIFO this is a great way to verify the specification and run tests, as memory of the state where the FIFO is at is particularly important to make sure that, for example, the reads are in the same order as the writes, what number of items are inside the buffer and what flags should be set and when.

Moving on to the second point, functional coverage was implemented to make sure the most extreme cases had been tested. This included checking if all inputs and outputs had been toggled and also 5 different cross cover

statements that represent different corner cases: we and full, which checks for the case of attempting to write to the full FIFO; we and empty, which checks for a write attempt when empty; we and re, which checks for a same cycle read and write; we and almost_full and re and almost_empty, a bit redundant but provide more confidence about the corner cases. While other combinations could have been tried these were identified as the most important corner cases and thus were the only ones included. This also solves points three and four, because if a test does not add additional functional coverage, then chances are it is redundant. Adding to point three, for a FIFO it doesn't really matter what we are writing to it. If the FIFO is 12-bit wide, there is no need to write 2^{12} different values to a single memory location, but rather just write any value to as many addresses as possible.

Three different tests were implemented to perform the full verification. Test 1 performs two write and read operations after a reset by writing a value and then reading it twice. This should verify simple behaviour from the FIFO. Test 2 starts by writing the FIFO until full capacity, then attempting to write two more values beyond that point. After these write attempts, the test performs a read and a write, in this order, for DEPTH+2 times to test, how the FIFO's pointers handle wrap arounds and then all entries of the FIFO are read, together with an empty read attempt. This test covers many edge cases and is the longest out of all. Finally, test 3 verifies same cycle read and write operations by writing a value and after that setting we and re to 1 and trying to write and read at the same time, finishing with a single read to make sure all items get read. It is important to note that in between all tests a reset of the FIFO is performed so that it goes back to a known state.

V. Results

This section presents the results of the testbench, simulated using ModelSim. The tesbench outputs to the transcript of ModelSim throughout the whole simulation. Every clock cycle, each module in the testbench writes log messages with information about their internal state which are triggered when certain tasks get completed. The scoreboard prints at the start of the next clock cycle the pass/fail results, which get added together and displayed at the end of the test as a final report. When all the tests are performed at the end of the simulation, the scoreboard produces a final report containing the number of tests that passed, a general pass or fail and the functional coverage percentage.

Figure 2 contains the results from the 3 tests performed together with the final report and the functional coverage statement from ModelSim.

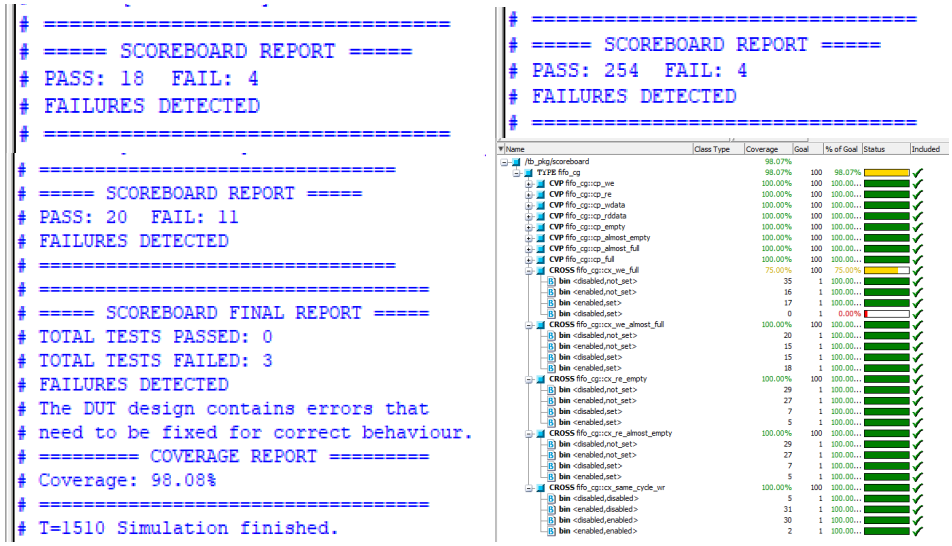


Fig. 2: ModelSim screenshots of simulation output. Starting at the top left and going clockwise: Test 1 result, test 2 result, test 3 result and final report, functional coverage report.

For test 1, the scoreboard logs showed fails at 4 points, but all coming from an initial error when the empty flag was still set one clock cycle too late which affected all later reads and delayed the FIFO. In the second test, three of the fails were due to mismatches in the empty flag, similar to the fails in test 1. However, another error was found of an incorrect read at one of the last clock cycles of the test. For the last test 11 fails were detected which

account for close to 33% of all the pass/fail checks performed by the scoreboard. These errors were again initially due to a wrong assignment of the empty and almost_empty flags and later for incorrect read operations.

The coverage report contains the total coverage together with the individual percentages of each coverpoint. The only bin that is not at 100% coverage is the cross cover between the disabled we signal and the Full flag set.

VI. Discussion

In this section the simulation results are analysed, starting from Test 1. This was a simple read and write operation after a reset where two values were attempted to be written to the DUT and read back. The scoreboard counted 18 passes and 4 fails, meaning a total of 22 checks performed. The first fail was an incorrect empty flag set after writing the first entry. Although a value had been written, the FIFO still had the empty flag set. This error propagated into future clock cycles as the next operation was a read and according to the specification when empty is high the read is undefined which was observed (rddata was an unknown value). The following operation was the second write and resulted in both the empty and almost_empty flags to go low, meaning 2 entries were inside the buffer (ALMOST_EMPTY_TH is 2), giving another fail as the almost_empty flag was still set in the reference model. On the last read, the rddata output was the first value, which proved that it had been written in the beginning after all. This means that the empty flag error most likely arises from incorrect logic and it shows that the FIFO does not behave like expected if written and read after being reset.

Moving on to test 2, a full write and read of the FIFO, together with a pointer wrap around, a write while full and read while empty tests were performed. This test was longer than the previous one but only got 4 fails with 254 passes. The first one was after the reset, where the pointers are reset and so is the empty flag, which was not true. The next clock cycle the error was the opposite, where the first value had been written and the empty flag was set, similar to the test 1 observations. There were no more fails recorded by the scoreboard until the end of the test. This verifies that the FIFO can be fully written to, doesn't enable write operations when its full, its pointers wrap around silently and at the right moments. The next fail only came when the penultimate entry was being read, which left a single entry in the FIFO and triggered the empty and almost_empty flags in the DUT. The final read came next, and while the reference got the expected value, the DUT was stuck at the past, meaning it hadn't updated rddata in that cycle, which occurred due to empty being set, although there still was a value inside it.

Test 3 checked if the FIFO complied with the specification of same cycle read and write operations. This test got 22 passes but 11 fails. It also started from a reset, and the first 2 fails were the same as test 2, since a value was written. The next operation was the first read and write cycle which due to the delay introduced by the wrong empty flag was also wrong. This was the case for every other cycle, where the read would be one value behind the expected. However, the FIFO was able to write the values and read at the same time, just not from empty and after a reset.

Finally, the functional coverage report for this simulation got a total of 98.07%, with 100% for all but 1 bin, which was the cross cover We disabled and Full is set. In this bin the FIFO is full with no writes happening, so no behaviour is verified. This means that all important cases were hit at least once, proving that the test was thorough.

The performed tests do contain some limitations like the reliance on the reference model, which may contain errors that could give false positives or even miss some errors altogether.

Future works for an even more thorough verification of this module could include tests of different parameter configurations, particularly with the DEPTH and WIDTH, since these are used for the logic for the flags and could expose more errors. Furthermore, while the reset was verified to restart the pointers, it wasn't fully checked if it was synchronous. But overall, the design does not pass the test due to issues with the empty flag.

VII. Conclusion

In conclusion, this verification task focused on verifying a given FIFO buffer by using a UVM like testbench that compares outputs of the DUT from generated signals to a reference model in a scoreboard. The test got a high coverage of 98.07% and ultimately the DUT did not pass the tests due to errors in the empty flag logic that affected normal read and write operation.

References

[1] Admin, "Verification," ChipVerify. https://www.chipverify.com/tutorials/verification [Accessed: Apr. 28, 2026]

[2] Mark Zwolinski, "Lecture 10: Testbenches", ELEC2301, University of Southampton, Southamtpon, UK, April 2026. [Lecture slides]